



Secured Application Development



Taiz University

Dr. Abdullmalek Alqobaty

**Associated professor
Computer Engineering**

Secured Application Development

Password Management



Cross-Site Requests

Password Management



Password Management

In this lecture,

- We will look at the **storage and management of passwords**, both for our users and accessing services like databases.
- Passwords are often the **weakest link in web application security**. **This is because they rely on humans to be secure**. We have two types of attack to consider:
 1. **Cracking a password**
 2. **Getting the password from a user**
- **Password management is a responsibility of the user**. We have little control over users. However, we can limit the cracking of a password and mitigate the damage from a user disclosing it.
- We will begin by looking at **brute-force attacks** on passwords and **how to store passwords encrypted** to reduce this risk.
- We will follow this by looking at **techniques we can apply to make user passwords harder to guess** and **provide users with a safe way to reset their password**.
- Finally, we will look at **keeping our passwords secure when using source code control**.



Password Management

Storing Passwords

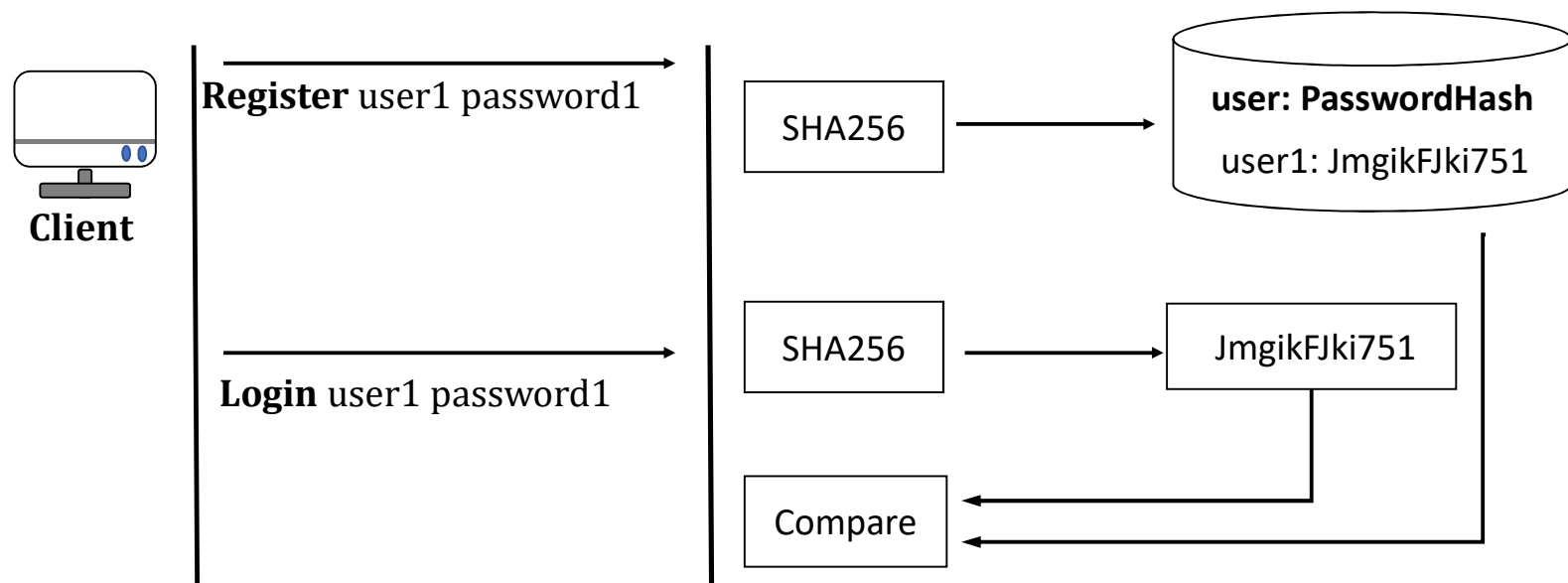
- ☐ We usually **store application passwords in a database** on the server. If we store it as plaintext and the password table is disclosed, an attacker gains immediate access to the passwords. **It is therefore more usual (but not universal) to store hashes of the passwords instead.**
- ☐ Hashing is normally used rather than encryption.
 - ☒ **If encryption is used, a key must be present to decrypt the password.** If the key is compromised, all passwords must be reencrypted, and accounts are vulnerable to attack in the meantime.
 - ☒ **Encryption is also unnecessary.** Instead, when a user enters a password to register, a hash is created from it, and the hash is stored in the database. **When the user logs in, the entered password is again hashed and compared with the stored version.**
- ☐ This is illustrated in Figure below. **Hashed passwords are more secure than encrypted passwords because they cannot be reversed,** even with knowledge of the algorithm.



Password Management

Cross Password Management

- We looked before at hashing algorithms. Common algorithms are MD5, SHA-256, and SHA-512. **MD5 is no longer considered secure enough for password hashing.** SHA-256 and SHA-512 are better options.





Brute-Force Attacks

- ❑ In a brute-force attack, the attacker systematically tries every password until one matches. **This can be done directly on the login page of an application using a script.**
 - ☑ The attacker first tries a, then b, up to z, digits, uppercase and punctuation, then aa, ab, etc., until all possible combinations have been tried.
 - ☑ The number of combinations will depend on the password length and the number of different characters.
- ❑ Brute-force attacks are much faster when passwords are simple words that exist in the dictionary, as only words need to be tried instead of all alphanumeric combinations.
- ❑ As a comparison,
 - ☑ there are over two hundred billion combinations of up to eight lowercase letters.
 - ☑ The full Oxford English Dictionary contains around 270,000 words, so trying only English words, even including obsolete ones, saves time by six orders of magnitude.
 - ☑ Cracking passwords this way is called a dictionary attack. *We will return to this later when we talk about password policies.*



Brute-Force Attacks

- ❑ To prevent brute-force attacks on the login page,
 - ▣ it is a good idea to artificially delay the response after a rapid series of failures from the same IP address.
 - ▣ As an added security, you may wish to block the user's account after further successive failures.
- ❑ If the attacker can get a copy of your usernames and password hashes,
 - ▣ brute forcing becomes simpler as the attacker can perform it offline without accessing your login page. Not only is this more discrete, evading detection, but it is also faster because no network is involved and the attacker can use multiple CPU cores and GPUs.
- ❑ Attackers use software such as **John the Ripper** or **THC Hydra**. These tools take a dictionary as input and try various common modifications of each word such as case change and appending digits.
 - ▣ Hackers can write custom modification rules based on their experience of common passwords. Therefore, adding digits or making small changes to words does not prevent dictionary attacks.



Rainbow Table Attacks

- Depending on password length, brute forcing can be very slow. *As many sites use the same hashing algorithm, hackers often use a rainbow table.*
- ☑ Rainbow table *is a precomputed table of hashed passwords generated using the brute-force approach but then stored on the attacker's computer as a map from hashed to plaintext password.*
- ☑ It still takes a long time to compute, but it only has to be done once. Then, once a password hash file is compromised, the hashes can simply be looked up in the rainbow table.
- Rainbow tables are large, but storage is feasible. *If stored unencrypted and unindexed, a table of all passwords of up to eight characters plus their SHA-256 hashes would consume around 8 TB.*



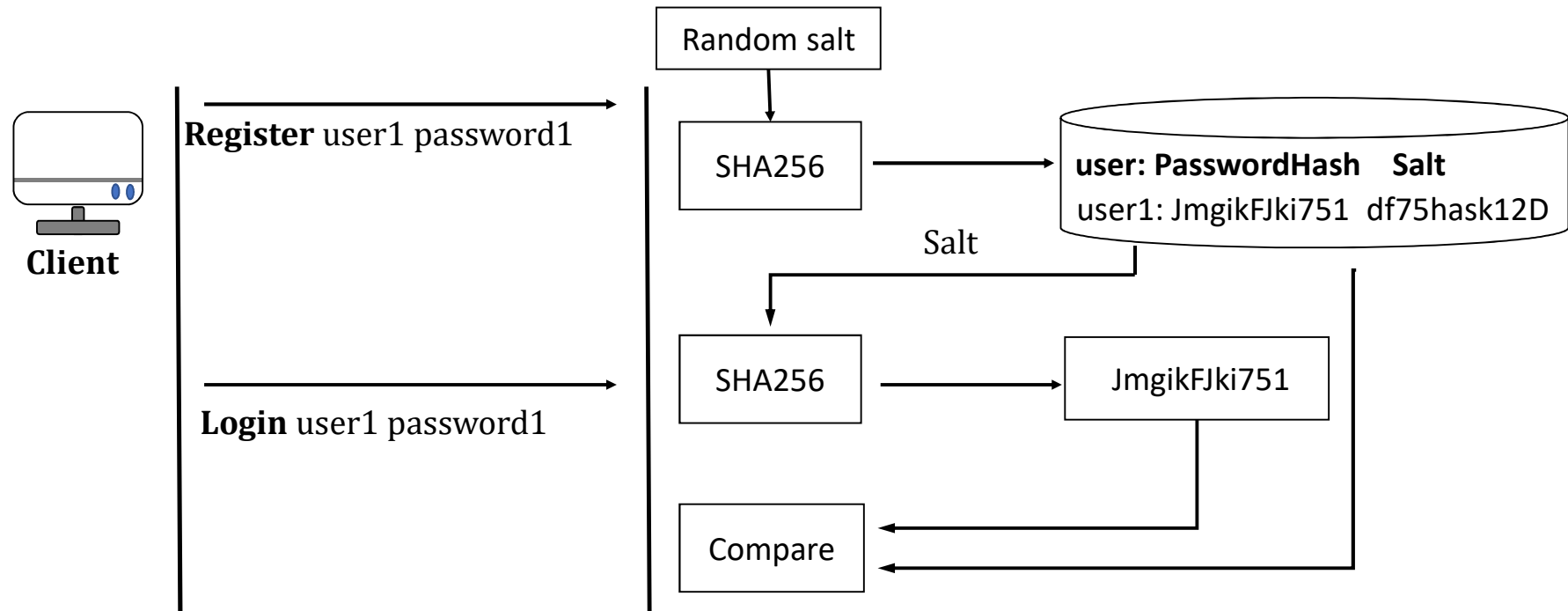
Salted Hashes

- ❑ To prevent rainbow table attacks, a salt is often used. This is a random string that is appended to the plaintext password before creating the hash.
 - ▣ As it is random, it has to be stored alongside the username and hashed password. The process is illustrated in the Figure below.
- ❑ A random salt, unique per user, is created when the user registers. This and the password are hashed, and the result is stored, along with the salt.
 - ▣ When the user logs in, the salt is fetched from the database and used to rehash the entered password. This is compared against the stored hashed password.
- ❑ Salting does not make brute forcing harder, other than making the algorithm a little slower because it is hashing a longer string.
 - ▣ As the salt is stored along with the hash, both are usually compromised together.
- ❑ The purpose of salting is to make rainbow tables infeasible.



Password Management

Salted Hashes





Salted Hashes

- Rainbow tables rely on a password always mapping to the same hash. When creating a rainbow table. Hackers enumerate all possible one to eight character passwords and create the hash for each one.
 - ☑ When a site's hashed password table is compromised, they look the hash up in their table and find the plaintext password that corresponds to it.
 - ☑ If the compromised hash were created from a salt as well as the plaintext password, the hash would not match anything in their table due to the salt.
- To guarantee their table has a match, they must create a hash of every password *and* salt combination. If the salt is eight characters, they need hashes of every combination of $1 + 8 = 9$ characters for single-character passwords, $2 + 8 = 10$ characters for two-character passwords, and so on up to $8 + 8 = 16$ characters for eight-character passwords.
 - ☑ There are now 4.5×10^{22} hashes to compute, and to store them uncompressed and unindexed needs 2.1×10^{24} bytes.



HMAC and PBKDF2 Password Security

- ❑ Salting addresses the threat of rainbow tables. As noted previously, it does not address the threat of brute-force attacks.
- ❑ The Password-Based Key Derivation Function 2 (PBKDF2) was designed to mitigate these attacks. It is described in RFC 2898 [15].
 - ▣ PBKDF2 is a password hashing algorithm based on a pseudorandom function (PRF), which abstracts out the hashing method from the algorithm.
- ❑ A common choice is Hash-Based Message Authentication Code (HMAC). This algorithm uses a hashing function such as SHA-256 plus a secret key to produce a cryptographic code. It is called a message authentication code because it can only be reproduced using the secret key, providing message authenticity.
- ❑ HMAC provides additional security, the secret key, to generate the same hash. As several hashing algorithms can be used with HMAC, HMAC algorithms are denoted as HMAC-MD5, HMAC-SHA256, etc.



HMAC and PBKDF2 Password Security

- ❑ The goal of HMAC is to make the hash that is a function of a secret key **K** as well as the plaintext message **P**. If the hashing algorithm is **H** (SHA-256, MD5, etc.), **the HMAC code is defined as:**

$$\text{HMAC}(K, P) = H(K \oplus \text{opad} || H(K \oplus \text{ipad}) || P)$$

Here,

- $\oplus$ is the XOR operator, and $||$ is the concatenation operator.
 - The value **opad** is the outer padding, consisting of **0x5C** repeated to the hashing block length B.
 - The value **ipad** is the inner padding, consisting of **0x36** repeated to length B.
- ❑ The equation looks more complicated than it really is. The padding serves the purpose of lipping half the bits of K. The reason for doing this is that HMAC is based on another secure PRF called NMAC, which is a function of two keys:

$$\text{NMAC}(K, P) = H(K_{\text{out}} || H(K_{\text{in}}) || P)$$



HMAC and PBKDF2 Password Security

- The XORing function creates the two keys K_{out} and K_{in} from the one key K . **The security of NMAC was found not degraded by the two keys being related.**
 - ▣ If K is longer than B , it is first hashed using the same algorithm H .
- The PBKDF2 password hashing algorithm is a function of a PRF,
 - ▣ along with the plaintext password P ,
 - ▣ a salt S ,
 - ▣ the number of iterations c ,
 - ▣ and the key length $dkLen$.
- It produces a derived key DK or length $dkLen$ octets:

$$DK = \text{PBKDF2}(\text{PRF}, P, S, c, dkLen)$$



HMAC and PBKDF2 Password Security

- ❑ In the case of HMAC, the PRF is a function of a key K as well as the plaintext password:
 $\text{PRF}(\bullet) = \text{HMAC}(K, \bullet)$
- ❑ The purpose of multiple iterations (the c parameter) is to make the **algorithm slower while still fast enough** to not impact legitimate users. **Slower algorithms are more time-consuming to crack by brute-force or dictionary attacks.**
- ❑ The key DK is made $dkLen$ octets long by performing a number of iterations l and concatenating the results.
- ❑ The algorithm works as follows.
 - ⊗ First, a hash is created by applying the PRF to a concatenation of P , S , and the iteration number i starting at 1 and ending at l . This in turn is performed c times.
 - ⊗ The l resulting keys are concatenated to form the resulting $dkLen$ -byte password hash.
- ❑ PBKDF2 mitigates brute forcing by
 - ⊗ Making the hash a function of a secret key
 - ⊗ Making the hashing function slow (OWASP recommends a minimum of $c = 310,000$ iterations)



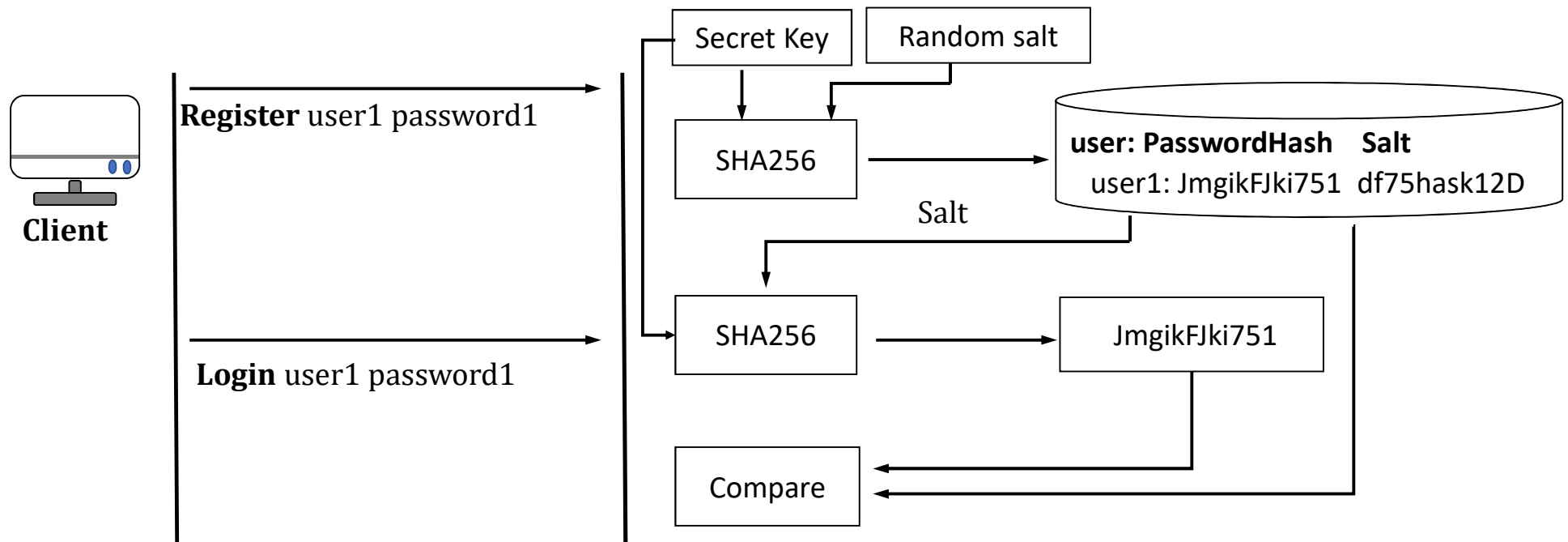
HMAC and PBKDF2 Password Security

- It also creates a fixed-length hashed password, meaning an attacker cannot use the length of the hash to guess the length of the password.
- **To brute force a compromised hashed password, the attacker needs the hash, salt, secret key, as well as knowledge of the algorithm (including c).**
- The login process using PBKDF2 and HMAC is illustrated in Figure below.
 - ☑ **When the user registers**, the password, a random salt, and the secret key are used to generate the hashed password to be stored in the database along with the salt.
 - ☑ **When the user logs in**, the salt is fetched from the database, and the typed password and secret key are used to create a hash to be compared with the stored hashed password.
- As a web application developer, we can use PBKDF2 without necessarily having to understand the preceding equations. **Django, makes it quite transparent to the developer.**
 - ☑ However, understanding how the algorithm works is important for understanding how it mitigates risks. Also, its complexity helps illustrate why developers should use an existing algorithm ideally implementation, instead of creating their own.



Password Management

HMAC and PBKDF2 Password Security





Password Policies

- Users like using real words as passwords. They also like using their username, real name, and partner, children, and pet names. In an attempt to make them more secure, they may add digits or substitute characters.
 - ▣ Hackers know this and systematically try modifications of dictionary words, starting with the most common.
- This particular task is aided by databases of compromised passwords. *These databases, consisting of 1–8 billion passwords, are available online.* They are useful to hackers for the following reasons:
 - ▣ The password for an account may already be in the database, avoiding the need to use brute force.
 - ▣ Hackers can derive a set of common passwords that are not in the dictionary.
 - ▣ Hackers can learn common rules that users employ to modify dictionary words (such as adding digits).



Password Policies

- We saw earlier that there are around two hundred billion passwords of one to eight lowercase letters. **The actual number is 2.2×10^{11} .** If we add uppercase letters and 12 special characters (for a total of 64 different characters), **that number increases to 2.8×10^{14} combinations.**
- This means passwords are over 1,000 times harder to brute force. **If we add nine character passwords, the number increases to 1.8×10^{16} . This is 100,000 times harder to brute force than passwords with one to eight lowercase letters.** Passwords of eight characters used to be considered sufficient, and in fact old hashing algorithms such as Unix crypt could only hash passwords of no more than eight digits.
- With modern cracking speeds, nine characters is more often considered a good minimum. **The longer the password is, the stronger it is.**



Password Policies

- ❑ Checks for password quality can be performed when the password is created. Some common rules are as follows:
 - ☑ Password has a minimum length (e.g., nine characters).
 - ☑ Password contains lowercase letters, uppercase letters, and symbols.
 - ☑ Password is not similar to the user's username or real name.
 - ☑ Password is not solely a dictionary word.
- ❑ Organizations often implement password expiration policies. These require users to change their password after a certain amount of time has elapsed (one year, three months, etc.) **This practice is now discouraged as it is no longer believed to increase security.**
- ❑ Users are likely to make only slight changes to their passwords. They are also more likely to write them down.



Password Reset

- ☐ A common feature on web applications is to allow users to reset their passwords when they forget them.
- ☐ The application needs to confirm that the request is made by the owner of the account.
 - ☒ A common pattern is to email a token to the user's registered email address, or phone number as legitimate ways.
- ☐ The token may be part of a URL that sends the user to a page to reset the password, or it may be a code that is entered manually. **The token should satisfy the following:**
 1. The server should be able to confirm it belongs to a registered user.
 2. The token should be a function of the user (username, ID, or Email).
 3. The token should be difficult to brute force.
 4. The token should expire after a short time elapses (e.g., one hour or one day). It should also be one-time only.



Password Management

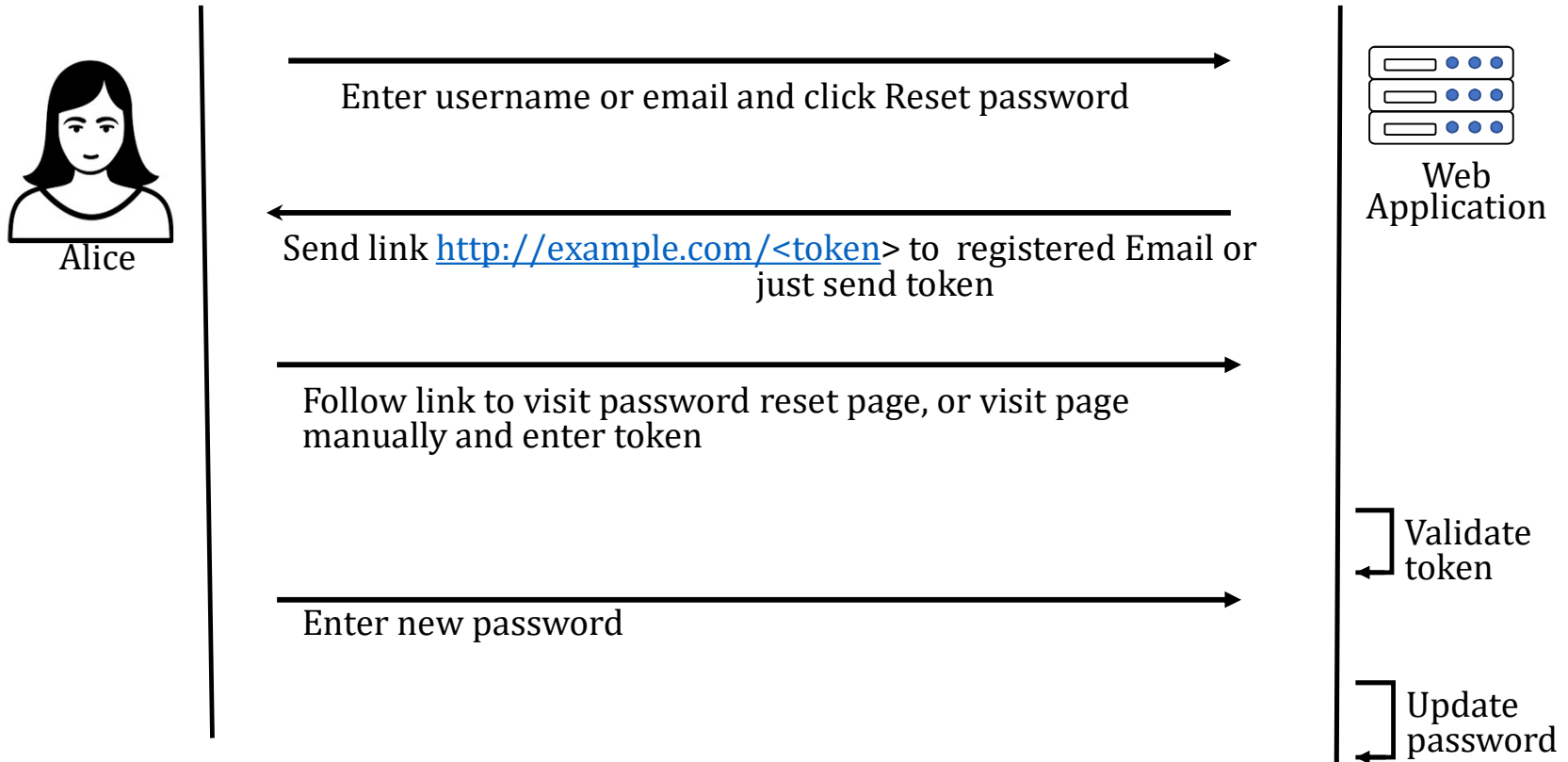
Password Reset

- A simple workflow is illustrated in the Figure below. It proceeds as follows:
 1. The user visits the password reset request page, enters username or email address, and clicks the Reset button.
 2. The server generates a token and sends a link, to a reset page with that token as a GET parameter, to the user's registered email address. **Alternatively, it just emails the token.**
 3. The user follows the link or visits the password reset page and manually enters the token.
 4. The server validates the token and extracts the user ID. If it is a valid token and has not expired, ask the user to enter a new password.
 5. The server changes the password for that user ID.
- We saw that GET requests should not change state in order to avoid CSRF attacks.
- The preceding GET request does not actually change state: it takes a user to a page that prompts them for a new password. **The new password is sent in a POST request.**



Password Management

Password Reset





Password and Source Code Control

- ❑ **An easy mistake to make is saving secrets**, such as database passwords and secret keys, in the source tree.
- ❑ **Another easy mistake is to recognize the error, remove them, and commit the change.** Your password will still be in the revision history, and removing it completely is a more messy undertaking.
- ❑ **One way to avoid this mistake is to keep passwords separate from code**, for example, in a service such as HashiCorp's Vault.
- ❑ **Another simple and popular method is as follows.**
 - ☞ We will assume Git as the source code repository, but it applied to others as well. Put all your secrets in one file, say, config.env. This file should be read into settings.py and anywhere else where secrets are needed.



Password and Source Code Control

- ❑ **One option is to make it a Bash script, for example:**

```
export DBPASSWORD=mydatabasepwd
export SECRET_KEY=djangosecretkey
```

- ❑ **Ensure this file is sourced before your application code runs.** In settings.py, you can refer to environment variables as follows:

```
import os
...
SECRET_KEY = os.environ['SECRET_KEY']
```

- ❑ **We use this technique in the Coffeeshop app.** The file is /secrets/config.env.
 - ☞ To make the variables in it available to Django, we source it into Apache with the following line in /etc/apache2/envvars:

```
. /secrets/config.env
```



Password Management

Password and Source Code Control

- ❑ **The environment file should not go into Git** so we can put an exclusion in the `.gitignore` file. Simply `config.env` on a line by itself should suffice, or prefixed with its subdirectory relative to the Git folder.
- ❑ So that your developers know what format this file should be, create another called `config.env_template`. Set the same variables in it, but to dummy values. When a developer clones the repository, they should copy `config.env_template` to `config.env` and change the values for each variable. The same should be done in production (obviously with different values). The `config.env_template` file can go into Git as it contains no real passwords.
- ❑ Note that in Coffeeshop, we put too many secrets in `config.env`. The database password legitimately belongs here as the application needs it. The usernames and passwords for admin, bob, and alice would not be in this file in production, and they are not used by the application, just for seeding it with sample data.